Datamodels

Graph Explorer — API Reference

Contents

- Ontology (schema authority — `DEFAULT_ONTOLOGY`)
- Filter operators
- Exploration tree (the request body)
- Response DTOs
- Persistence (SQLite `explorations` table)

Ontology (schema authority — `DEFAULT_ONTOLOGY`)

Entity types with single inheritance (`parentId`), each with attributes; plus a set of legal relationships. Subtypes **inherit** their parent's attributes and relationships.

Entity types

id	label	parent	own attributes (dataType)
`personnel`	Personnel	—	firstName, lastName, title, division, location (STRING)
`system`	System	—	name, location (STRING), criticality (NUMBER)
`technology_system`	Technology System	`system`	stack (STRING) *(+ inherited System attrs)*
`business_system`	Business System	`system`	businessUnit (STRING) *(+ inherited)*
`vendor`	Vendor	—	name, contactEmail (STRING), headcount (NUMBER)

Relationships (`typeId` : from → to)

typeId	label	from	to
`OWNS`	owns	personnel	system
`USES`	uses	personnel	system
`REPORTS_TO`	reports to	personnel	personnel
`HAS_IMMEDIATE_REPORTEE`	has immediate reportee	personnel	personnel
`MAINTAINED_BY`	maintained by	system	vendor

`direction` `IN` uses a relationship in reverse (surfaced with an "(inverse)" label); `is_legal` checks forward and/or inverse against the type hierarchy.

Filter operators

`DataType` ∈ `STRING` | `NUMBER` | `DATE` | `BOOLEAN`. Valid ops per type:

DataType	Allowed ops
STRING	EQUALS, NOT_EQUALS, CONTAINS, STARTS_WITH, IN
NUMBER	EQUALS, NOT_EQUALS, GT, GTE, LT, LTE, BETWEEN, IN
DATE	EQUALS, NOT_EQUALS, GT, GTE, LT, LTE, BETWEEN, IN
BOOLEAN	EQUALS, NOT_EQUALS

Value shape by op (enforced by the `Filter` validator):

- List ops — `IN`, `ID_IN`, `BETWEEN` — require a **non-empty list**. `BETWEEN` requires **exactly two** `[lo, hi]`.
- All other ops must **not** be given a list.
- `ID_IN` is special: it pins node **ids** (a list of strings), not an attribute — use with ids from `/entities/search`.
- DATE values are ISO-8601 strings (`Z` accepted).

Examples:

```
{ "attribute": "division", "op": "EQUALS",  "value": "Y" }
{ "attribute": "criticality", "op": "BETWEEN", "value": [3, 5] }
{ "attribute": "location", "op": "IN",     "value": ["NYC", "LON"] }
{ "op": "ID_IN", "value": ["person-42", "person-99"] }
```

Exploration tree (the request body)

```
Exploration
├─ explorationId?  string   (server-assigned on save)
├─ name?           string
└─ root            Column
                   ├─ columnId          string   (unique across the tree)
                   ├─ entityTypeId      string   (ontology type id)
                   ├─ filters[]         Filter   { attribute, op, value }
                   ├─ selectedAttributes[]  string  (attrs to return)
                   └─ children[]        Edge
                                        ├─ relationship  { typeId,
direction=OUT|IN|BOTH }
                                        └─ column         Column   (recursive)
```

- The tree is **recursive**: each `Edge` carries a relationship + a nested `Column`.
- A `Column` = "a node type to match, filtered, with chosen attributes, plus outgoing branches."
- Each root→leaf path becomes a set of report columns in RETURN order.

Response DTOs

DTO	Shape	Returned by
`Page<T>`	`{ items[], page, size, total }`	entity-types, explorations list
`EntityTypeDTO`	`{ id, label, parentId?, hasChildren }`	entity-types
`AttributeListDTO`	`{ attributes: [{ name, label, dataType, filterable }] }`	attributes
`EntitySearchDTO`	`{ items: [{ id, label }], total }`	entities/search
`RelationshipListDTO`	`{ relationships: [{ typeId, label, direction, toEntityTypeId }] }`	relationships
`PreviewDTO`	`{ counts: { columnId: int } }`	preview
`ExplorationCreatedDTO`	`{ explorationId }`	create exploration
`ReportDTO`	`{ reportId, generatedAt, columns[], rows[], truncated }`	reports:generate
`ValidationErrorDTO`	`{ errors: [{ columnId?, field?, message }] }`	any 400 from an invalid tree

`ReportDTO` detail:

- `columns[]` = `ReportColumnDTO { columnId, entityTypeId, attributes[] }` in RETURN order.
- `rows[]` = list of `{ columnId: { id, <attr>: value, … } | null }`. A null cell means that branch didn't match for that row.
- `truncated=true` when the result hit `max_rows` (5000).
- CSV form (`?format=csv`): flat, header cells are `columnId.attribute` (with `columnId.id` per column).

Persistence (SQLite `explorations` table)

column	type	notes
`id`	TEXT PK	`exp-<hex>`
`name`	TEXT	optional
`body`	TEXT NOT NULL	the Exploration serialized as JSON
`updated_at`	TEXT	`CURRENT_TIMESTAMP` , used for list ordering (DESC)